



**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

TinyML - CNN Operations

Assoc Prof Nicholas Vun
**College of Computing
and Data Science**

Convolutional Neural Networks

Convolutional neural networks (CNN)

- most popular deep learning architecture
- ubiquitous in computer vision

To achieve higher accuracy

- general trend in is to use deeper and more complicated networks
 - ResNet : 152 layers

But these advances

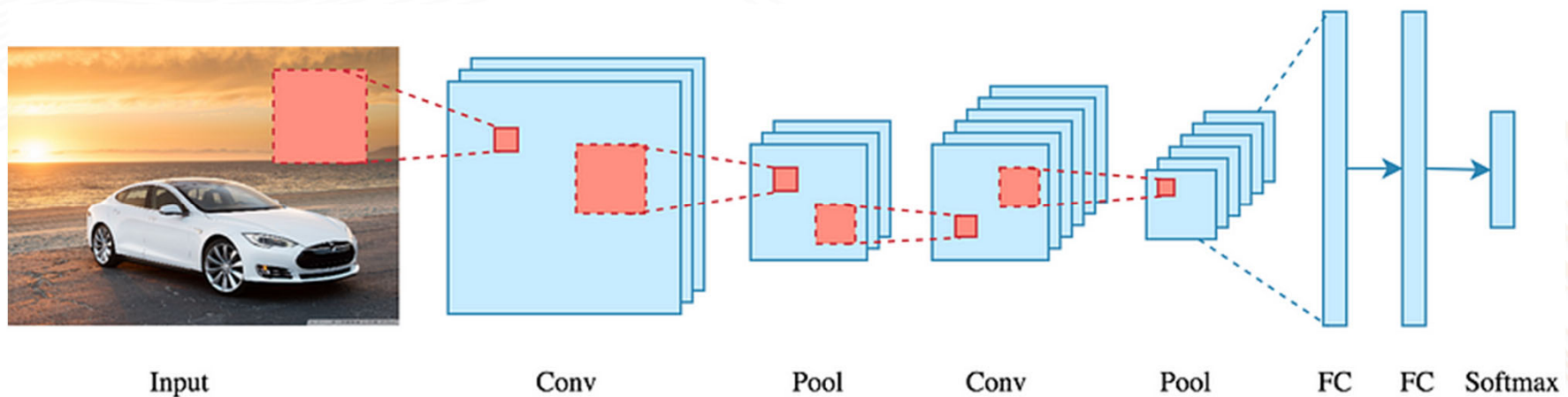
- may not meet the requirements of many real-world applications
 - where real time performance and limited model size are of major factors
 - E.g., autonomous vehicle, robotics and AR

Basis of CNN

CNN

- automatically detects the important features of image
- computationally efficient compared to fully connected multilayer perception (MLP) based DNN

Typical architecture: A series of convolution + pooling operations, with fully connected dense output layers.



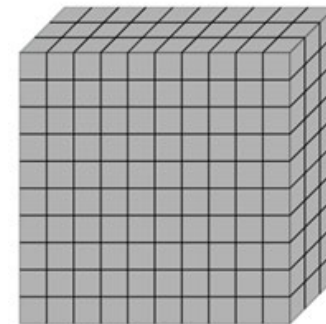
Convolutional Layer – Input Image

Main building block is the convolutional layer

- which consists of kernels (i.e. filters) applied to its input tensor
- to detect and produce features maps

Real world image is represented by a 3 dimension matrix

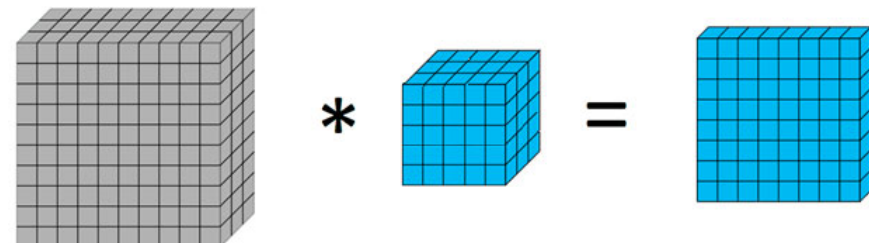
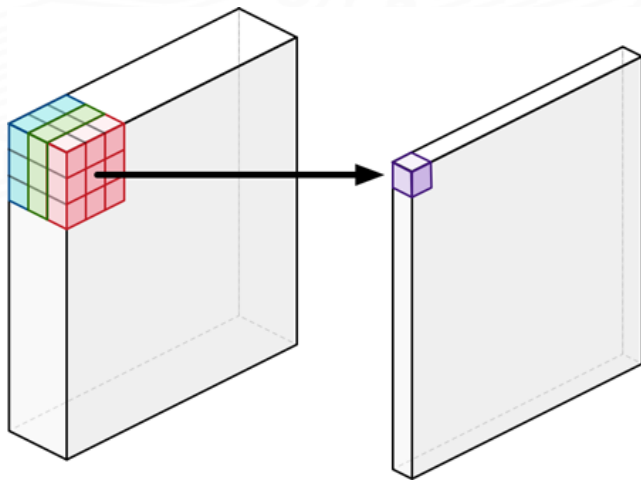
- height and width correspond to the image size
- depth corresponds to the 3 colours (RGB channels)



Convolution Operation

The convolution operation

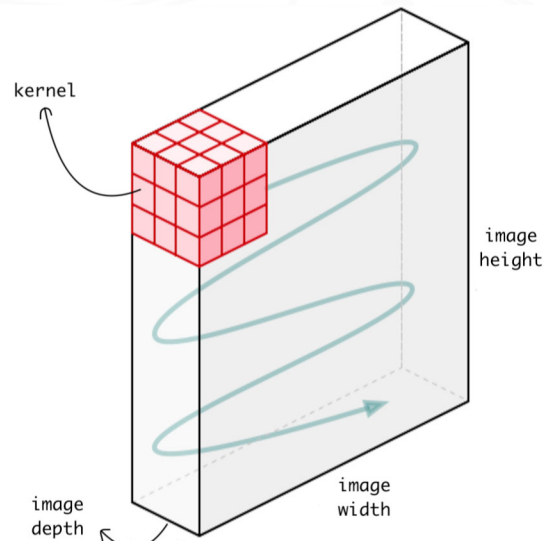
- a 3D kernel with specific height and width (e.g., 5x5) and covers the entire depth (i.e., 3 channels)
- slides across the input tensor to perform the cross-correlation dot-product calculation, and weighted summed together



2D and 1D Convolutions

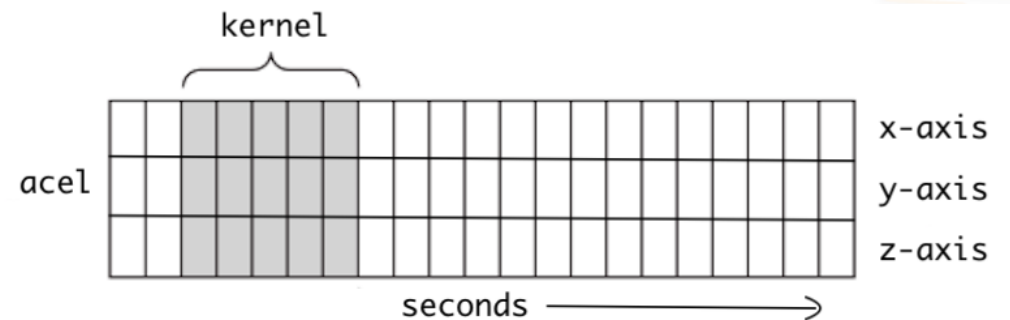
Conv2D

- 2 dimensional CNN because the **kernel** slides along 2 dimensions on the data
- generally used on image data



Conv1D

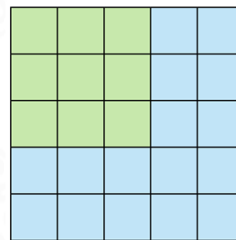
- kernel only moves in one dimension
- E.g. for time series data from IMU
 - along the axis of time.



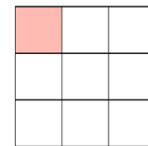
Convolution Operation – Stride

Stride

- the number of pixel the kernel is moved at each step
- E.g. stride of 1



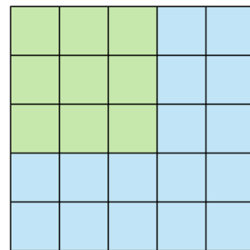
Stride 1



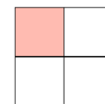
Feature Map

Bigger strides

- feature map become smaller
- E.g. stride of 2



Stride 2



Feature Map

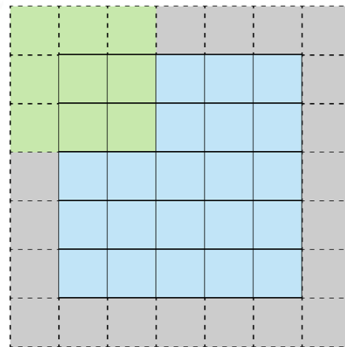
Convolution Operation – Padding

Stride within the input tensor reduces the size of the output tensor

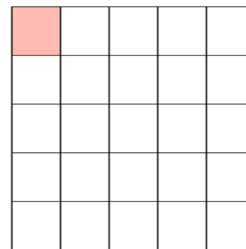
- feature map shrinks as it progress through the network
- which is not desirable

Padding is use to retain the dimension and preserve the feature maps

- surround the input tensor with zeros or the values on the edge



Stride 1 with Padding



Feature Map

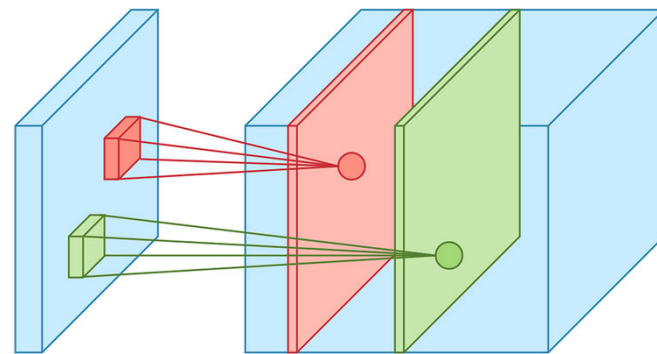
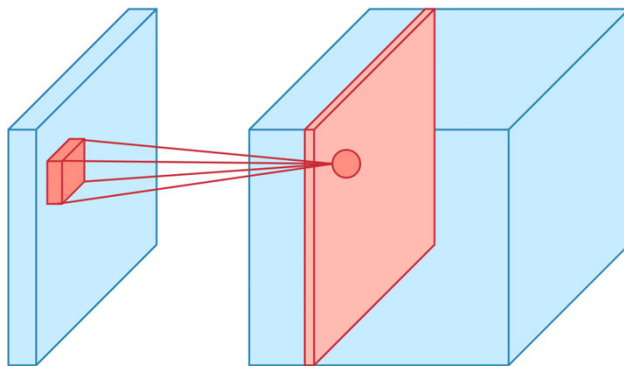
Multiple Kernel

In practice

- multiple kernels are applied to the input tensor
 - each is used to detect certain independent feature of the input

Each kernel produce one feature map

- multiple kernels produce multiple layers of maps, which are stacked together
 - depth of the output map increases as more kernels are used



Pooling Operation

After each convolution

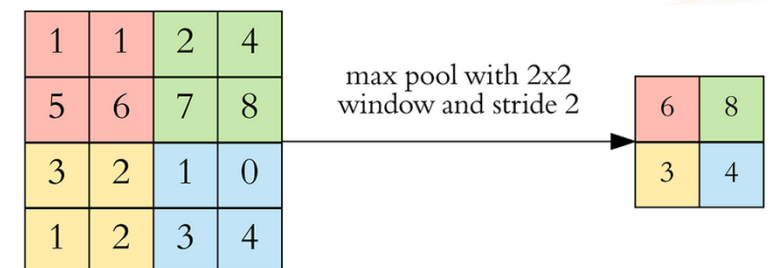
- the output tensor is applied to a non-linear function
- E.g. the ReLU activation function

Pooling is then used to reduce the dimension of the output tensor

- reduces the parameters and shortens the training time
- but retains the important feature information

Most common type: max pooling

- slide a window over the output tensor
- take the maximum values in the window



Pooling Operation (cont.)

Example:

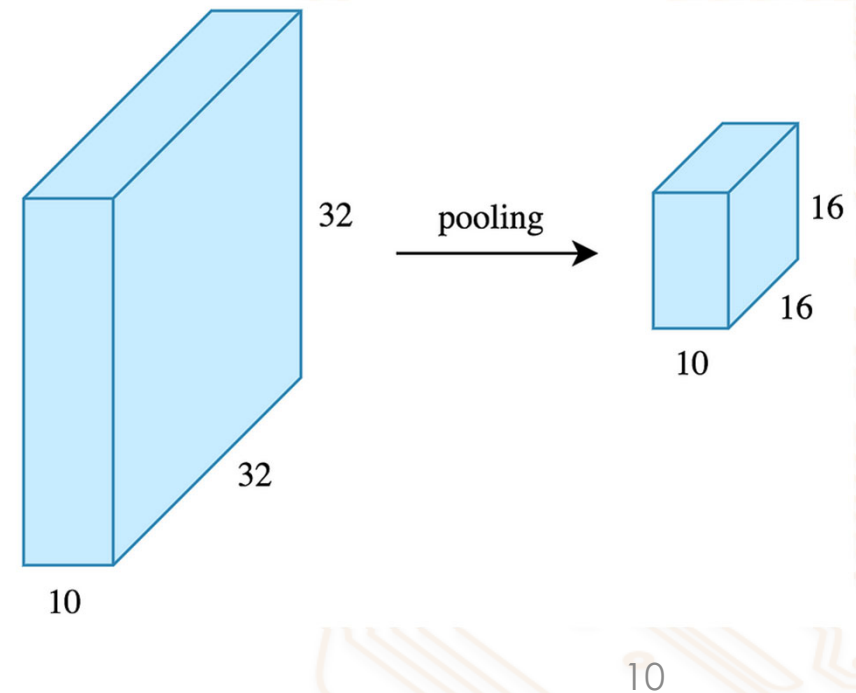
- a 10-layer 32x32 input tensor
- a 10-layer pooling with 2x2 windows and stride of 2

Stride of 2 will reduce the input tensor by two times (from 32 to 16)

- but the depth (i.e. 10 layers) remain unchanged

Number of parameters (i.e. weights)

- reduces by 4 times

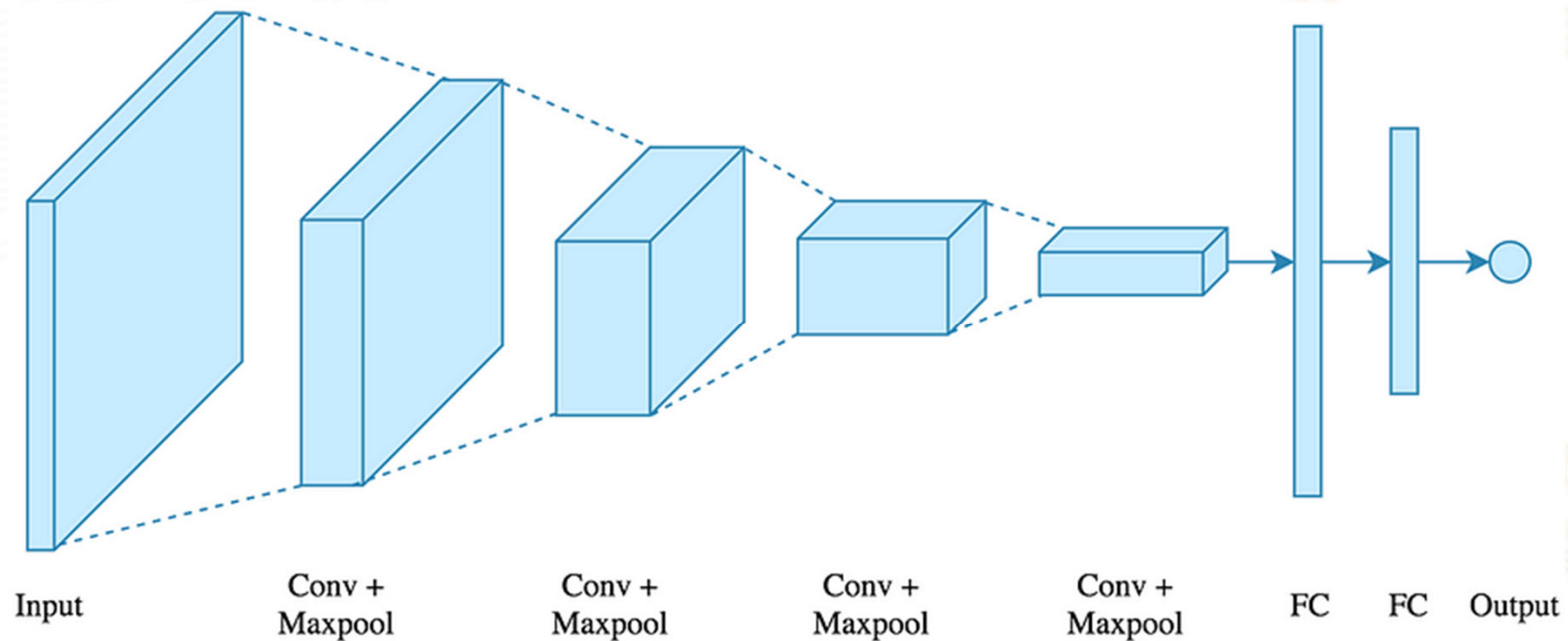


Overall CNN Architecture

After a number of convolution + pooling layers

- Fully connected layers are used to complete the CNN architecture

Example:



Examples of Kernels

In Image processing

- Filters (also known as Kernels) are used to detect patterns and features of the image

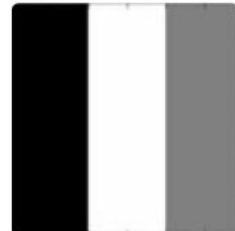
Example: In conventional Computer Vision based image recognition

- specific filters are designed to detect edges

-1	-1	-1
1	1	1
0	0	0



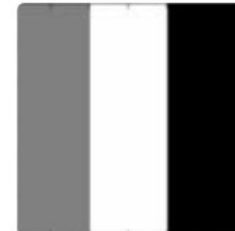
-1	1	0
-1	1	0
-1	1	0



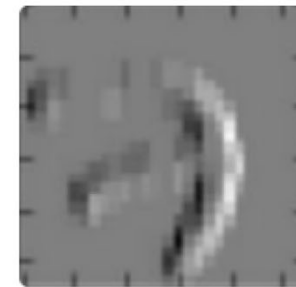
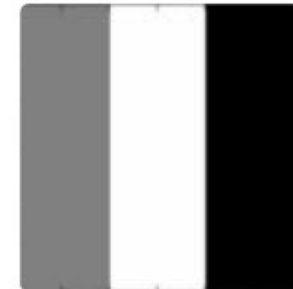
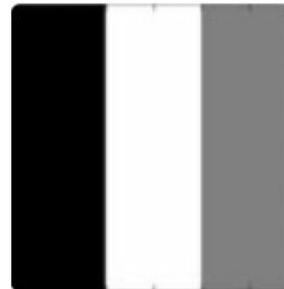
0	0	0
1	1	1
-1	-1	-1



0	1	-1
0	1	-1
0	1	-1



Case Study - Edge Detection Filters



Kernels in Machine Learning

In Deep Neural Network model training

- we let the machine **derives automatically** the most suitable filters to be used in the model

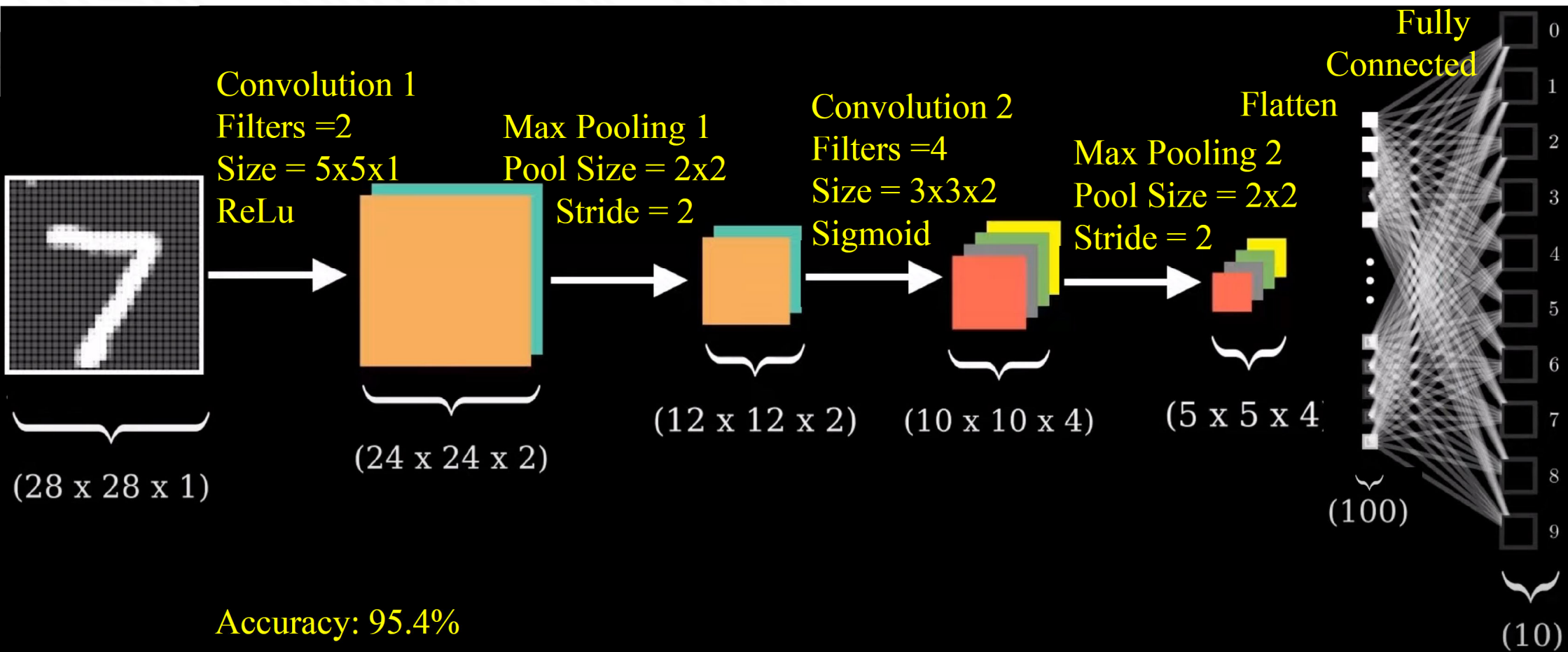
Each layer filters values start out with some random values

- usually normal distribution with mean of zero
 - with variance inversely proportional to the number of layers
- biases are set to zero initially

Filter values change as the network learns during training

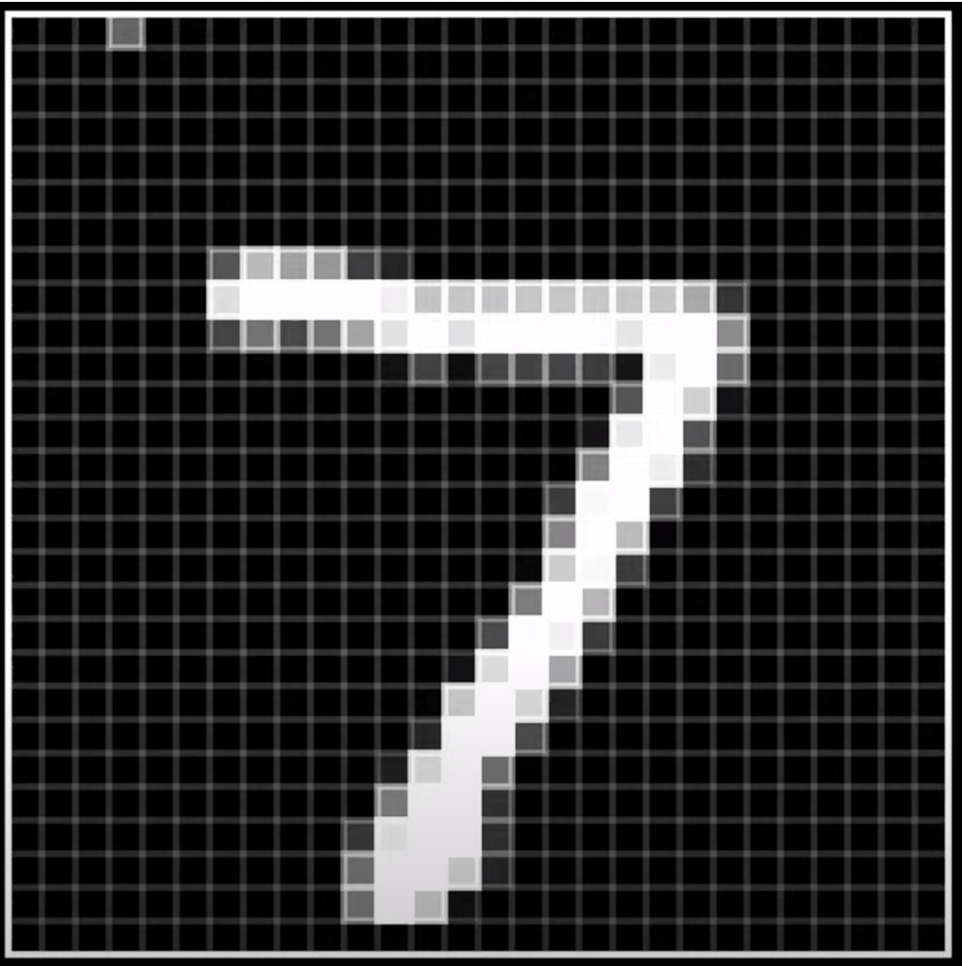
- such that the pattern detecting capability of the filters emerges automatically

Case Study: Architecture Details



(Source: www.youtube.com/watch?v=jDe5BAsT2-Y)

Convolution 1 – Two (5x5) Filters with ReLU AF

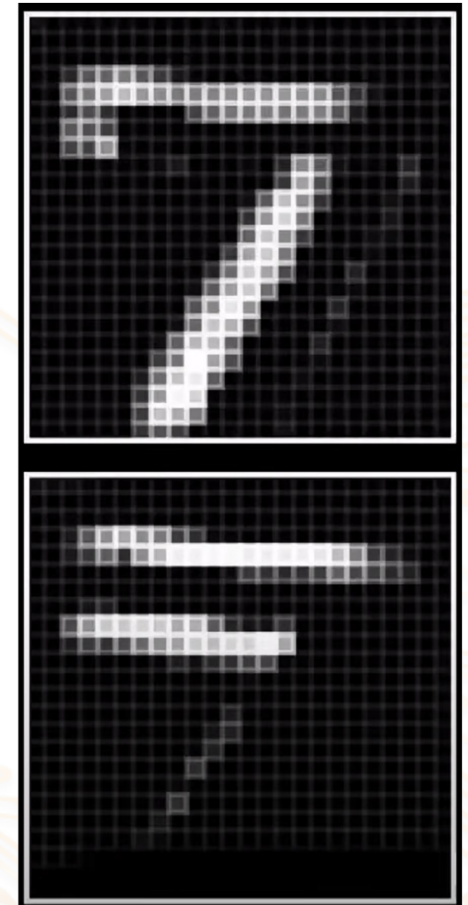


0.003	-0.077	-0.037	0.039	-0.095
0.103	-0.345	0.22	0.01	0.137
0.028	-0.259	-0.108	-0.077	0.14
-0.078	-0.062	0.021	-0.015	-0.018
-0.042	0.067	0.088	0.051	0.2

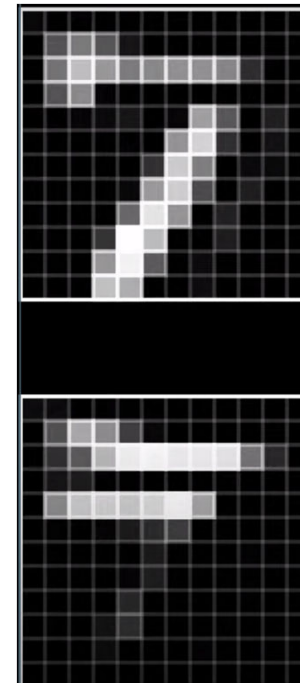
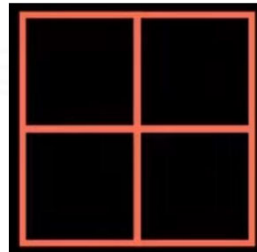
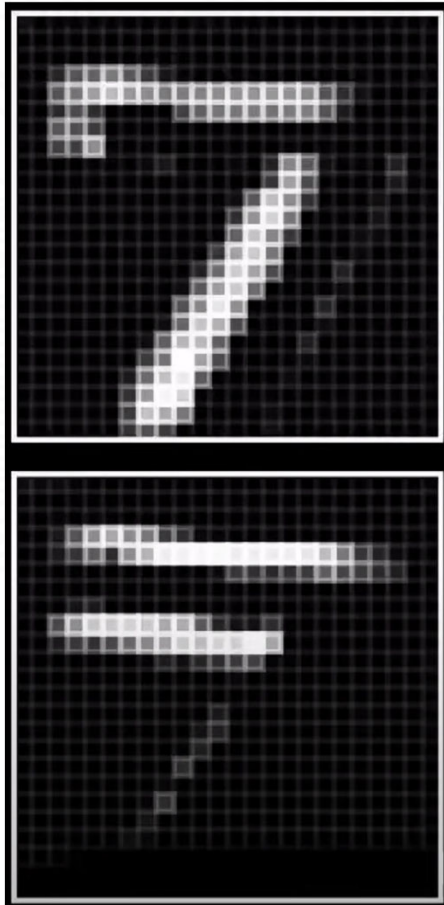
, 0.155

0.028	0.06	0.046	0.069	0.214
-0.093	-0.019	0.036	0.23	-0.115
-0.265	-0.43	-0.266	-0.278	-0.328
-0.005	-0.029	-0.244	-0.132	-0.216
0.111	-0.035	0.197	0.183	0.136

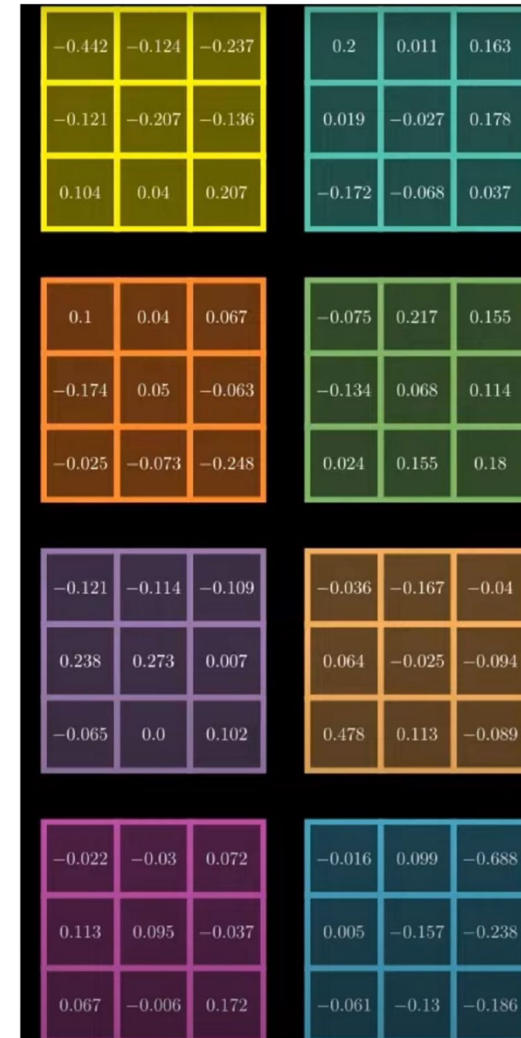
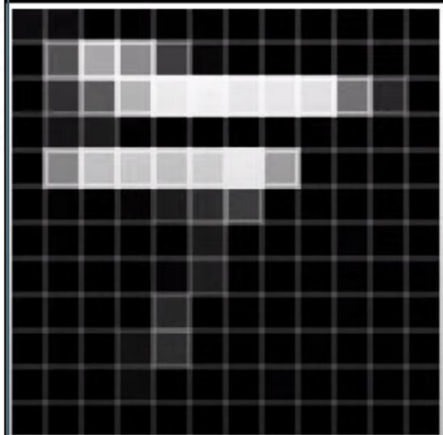
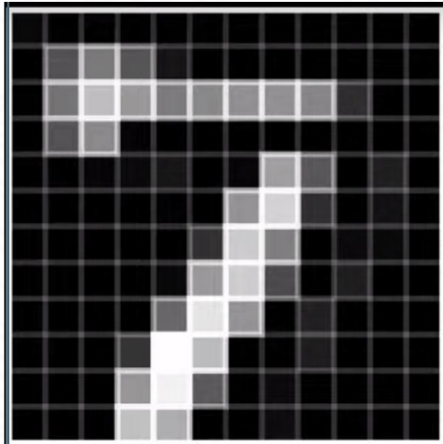
, -0.567



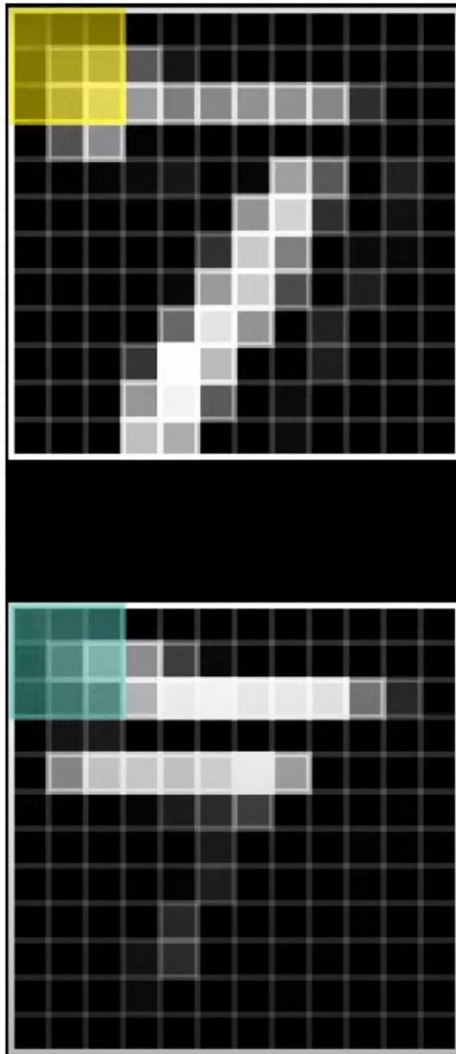
MaxPooling 1 (Stride 2x2)



Convolution 2 – Four (pairs of) Filters



Convolution 2 – 4 (pairs of) Filters with Sigmoid AFs

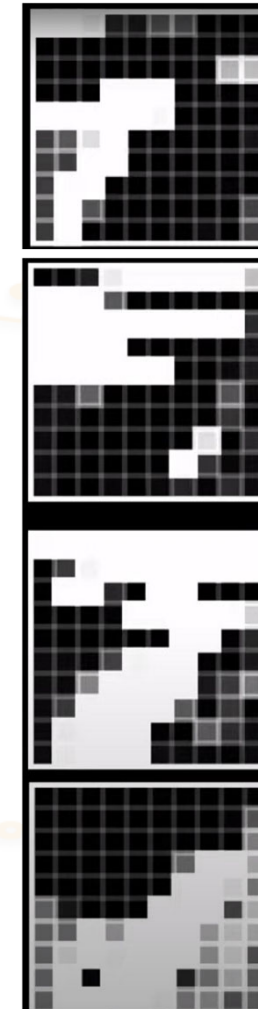


-0.442	-0.124	-0.237	0.2	0.011	0.163	, -1.069
-0.121	-0.207	-0.136	0.019	-0.027	0.178	
0.104	0.04	0.207	-0.172	-0.068	0.037	

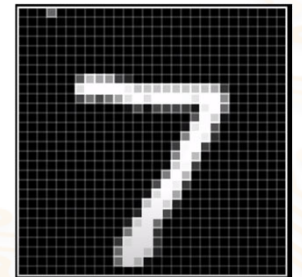
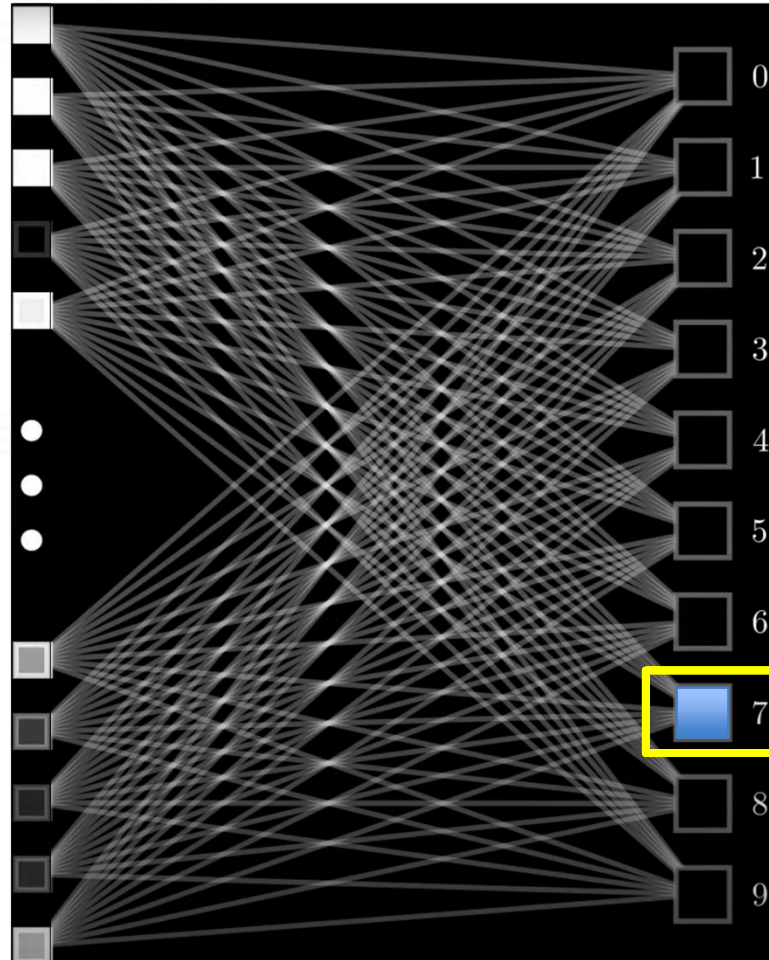
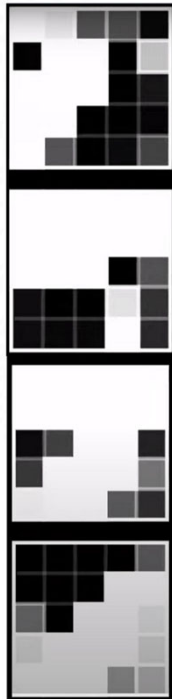
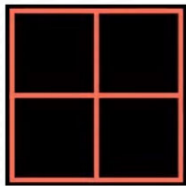
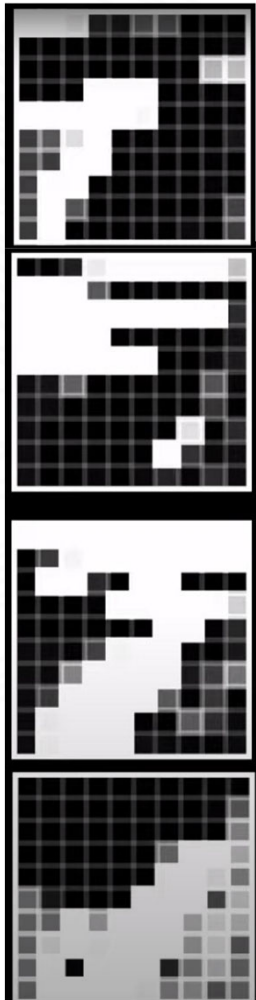
0.1	0.04	0.067	-0.075	0.217	0.155	, -2.87
-0.174	0.05	-0.063	-0.134	0.068	0.114	
-0.025	-0.073	-0.248	0.024	0.155	0.18	

-0.121	-0.114	-0.109	-0.036	-0.167	-0.04	, -2.963
0.238	0.273	0.007	0.064	-0.025	-0.094	
-0.065	0.0	0.102	0.478	0.113	-0.089	

-0.022	-0.03	0.072	-0.016	0.099	-0.688	, -0.212
0.113	0.095	-0.037	0.005	-0.157	-0.238	
0.067	-0.006	0.172	-0.061	-0.13	-0.186	



MaxPooling 2, Flattening and Fully Connected



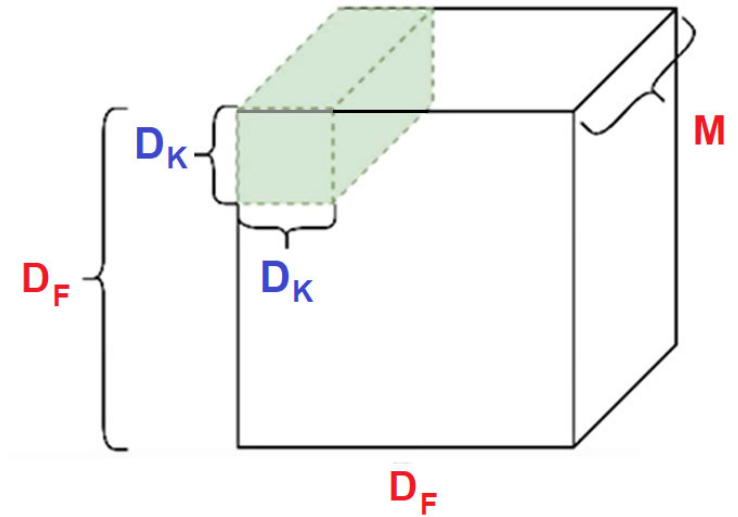
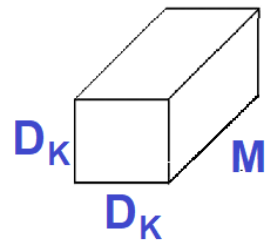
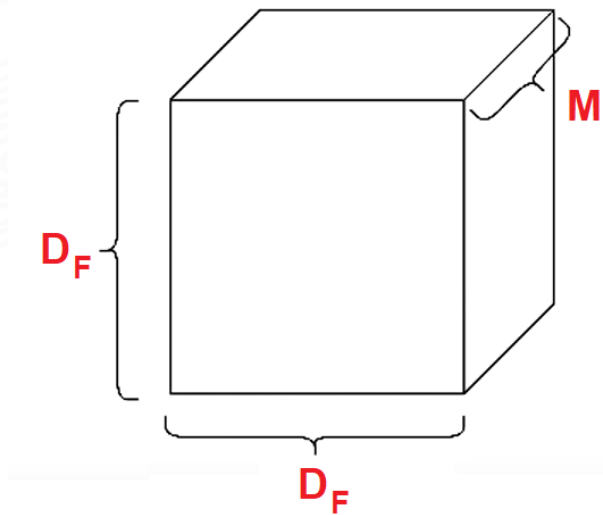
Computation complexity in CNN

We will now calculate the convolution cost

- in terms of number of multiplications

Consider an input tensor of $D_F \times D_F \times M$ (e.g., $20 \times 20 \times 3$)

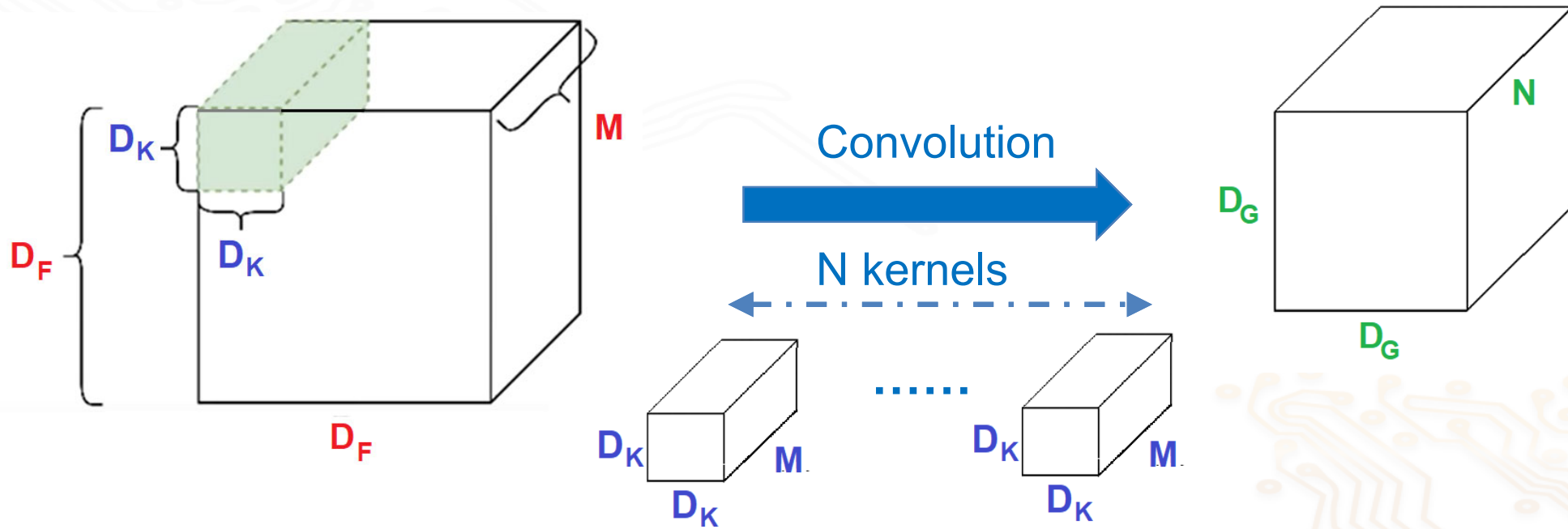
- a kernel size of $D_K \times D_K \times M$ (e.g., $5 \times 5 \times 3$)



Convolution Operation

When N kernels are applied to the input tensor

- size of output feature map = $D_G \times D_G \times N$
- D_G : depend on stripe and padding used



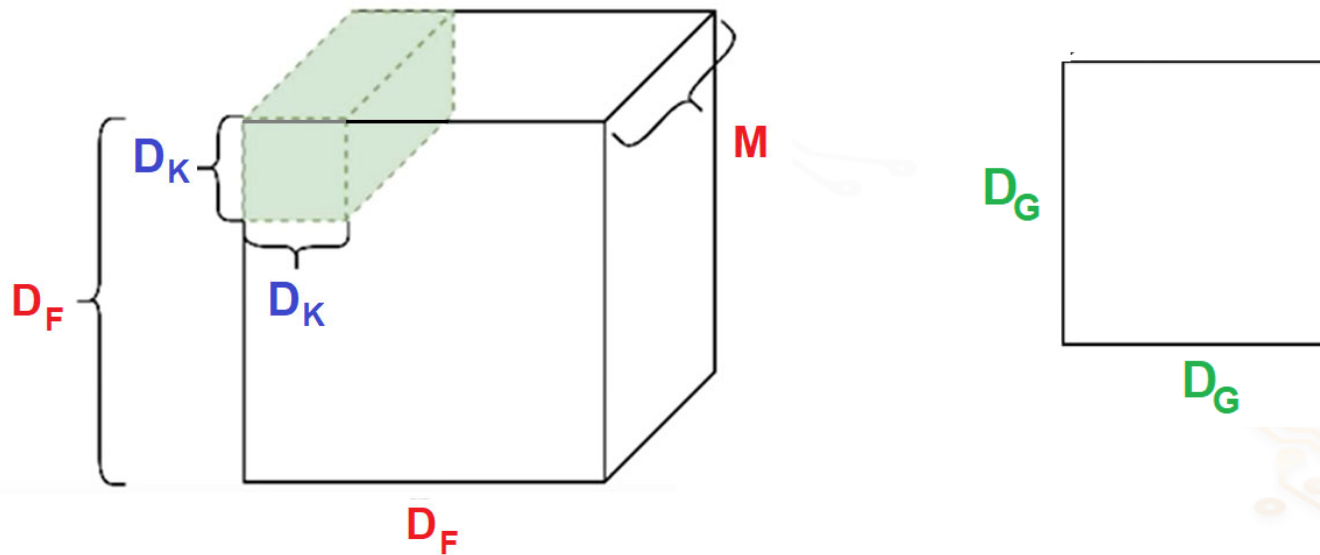
Computation complexity

Number of multiplication for one convolution

- $D_K \times D_K \times M$

As the kernel slides through the input tensor: $D_G \times D_G$ times

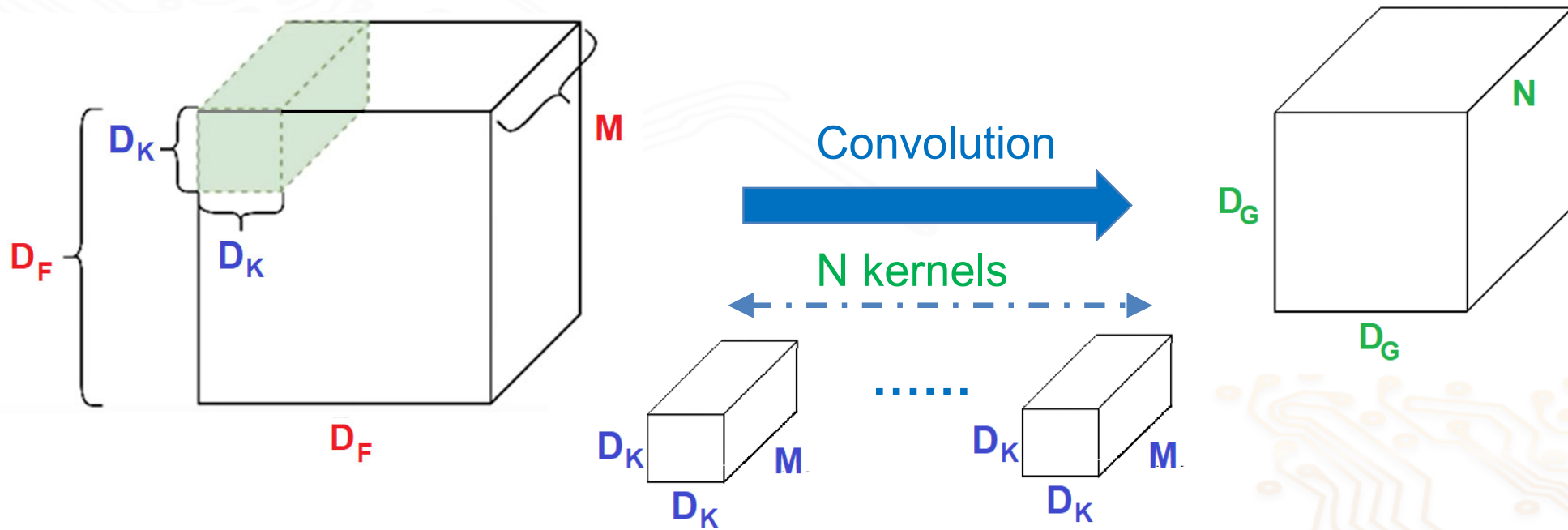
- Total number of multiplication **per kernel** = $D_K \times D_K \times M \times D_G \times D_G$



Computation complexity

With N kernels

- total number of multiplications = $N \times D_K \times D_K \times M \times D_G \times D_G$





**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

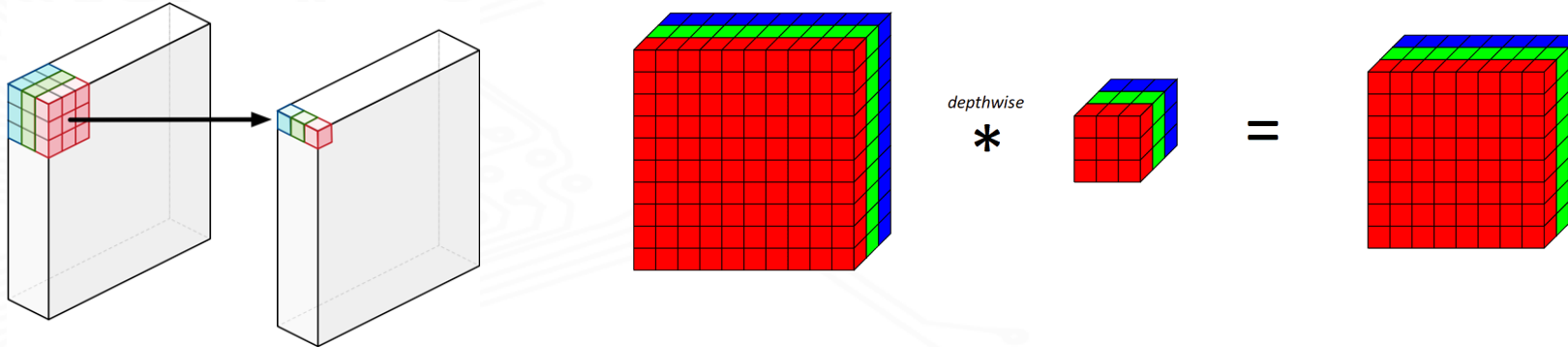
TinyML - Efficient CNN: Depthwise Separable convolution

Assoc Prof Nicholas Vun
**College of Computing and
Data Science**

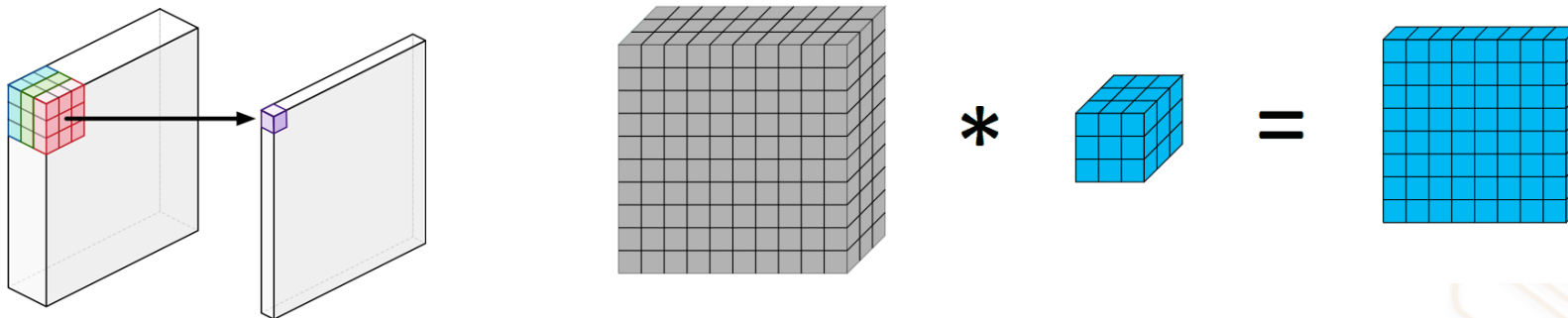
Depthwise Separable Convolution

Depthwise Separable (DWS) convolution is used to reduce CNN computational complexity

- by using separate convolution per channel



- c.f. conventional CNN

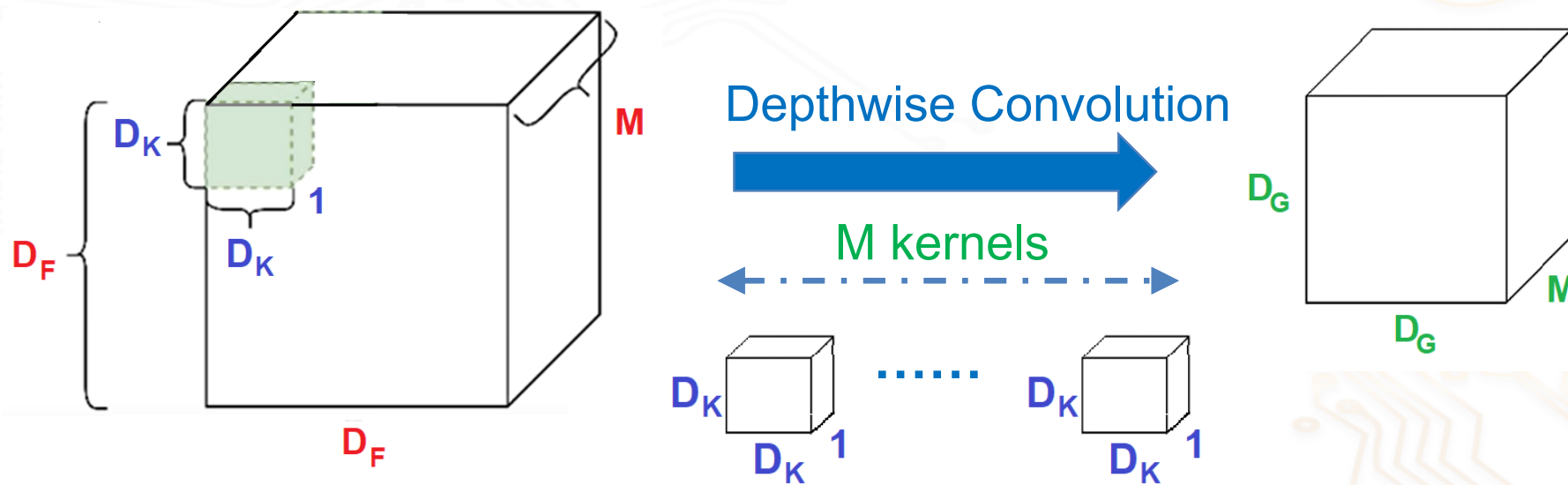
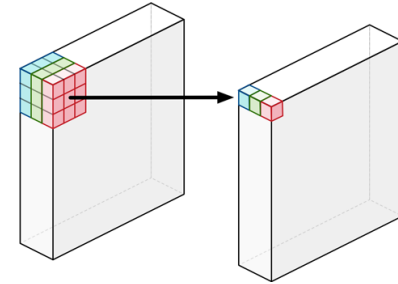


Depthwise Separable Convolution

There are **two stages** in Depthwise separable convolution

i) Depthwise convolution: Filtering stage

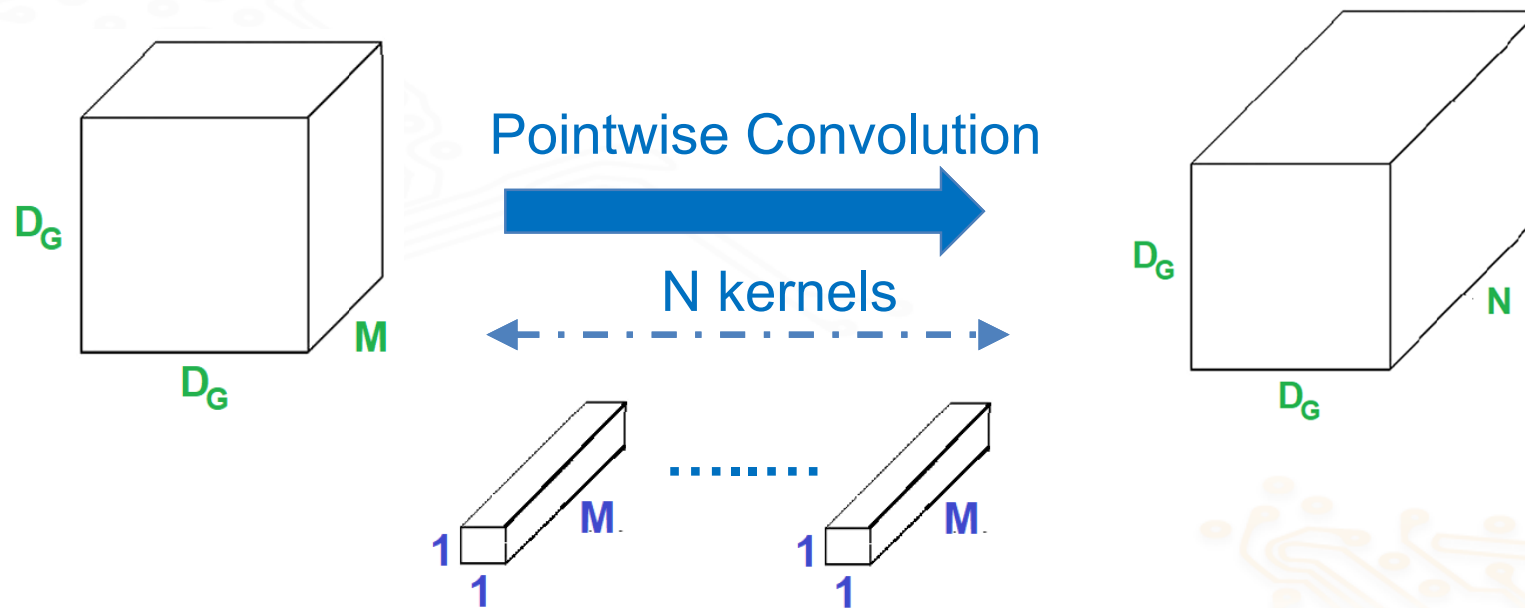
- input tensor is split into M separate channels
- each channel is convolved with a 2D kernel (i.e., $D_K \times D_K \times 1$)
- output of each channel is stacked together to form the output



Depthwise Separable Convolution

ii) Pointwise convolution: Combination stage

- tensor is convoluted with N kernels of size $1 \times 1 \times M$
- output of each kernel is stacked together to form the output

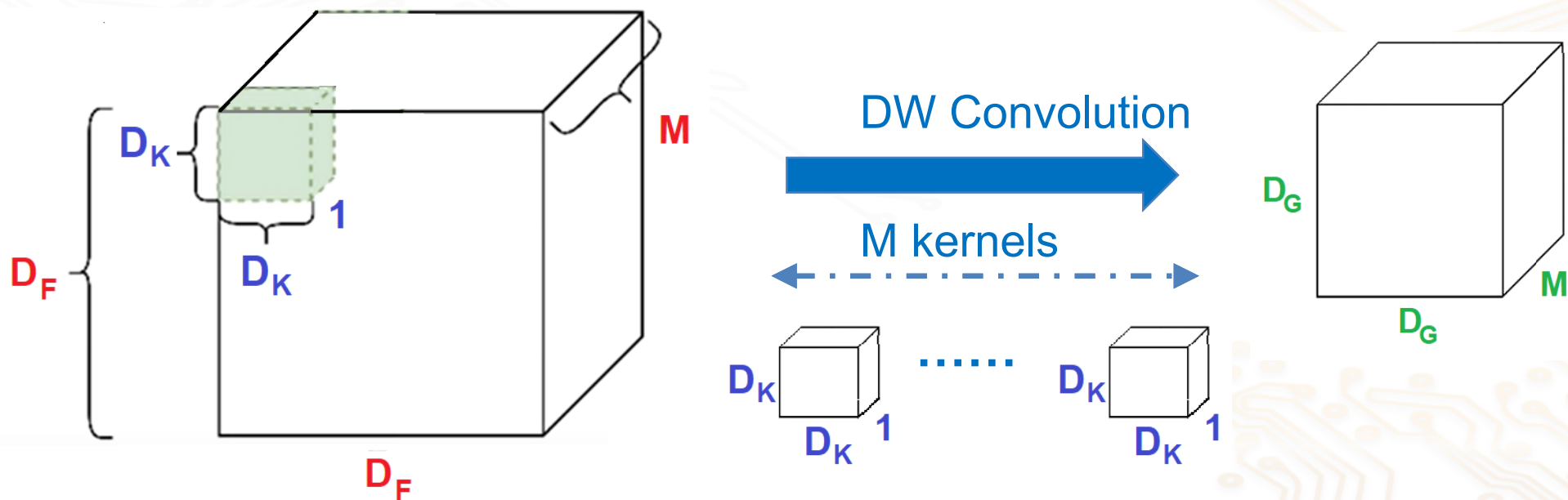


Depthwise Separable Convolution Complexity

i) Depthwise convolution

- number of multiplication per convolution per channel = $D_K \times D_K$
- number of multiplication per channel = $D_G \times D_G \times D_K \times D_K$

Total number of multiplication (M channels) = $D_G \times D_G \times D_K \times D_K \times M$

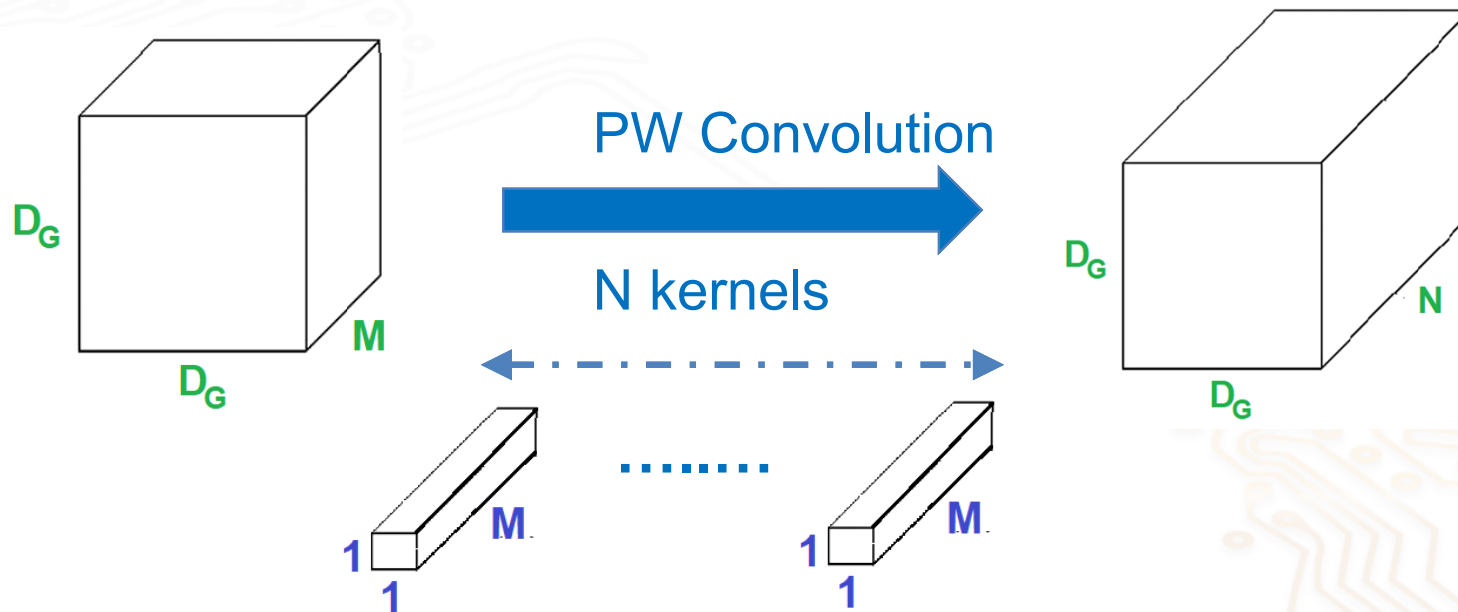


Depthwise Separable Convolution Complexity

ii) Pointwise convolution

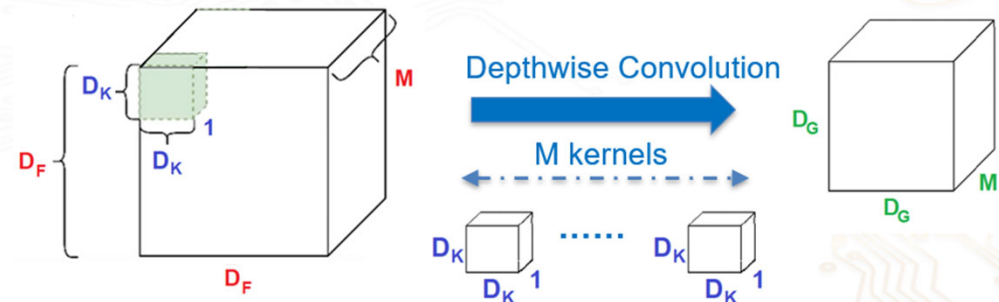
- number of multiplication per convolution = M
- number of multiplication per kernel = $D_G \times D_G \times M$

Total number of multiplication (N kernels) = $D_G \times D_G \times M \times N$

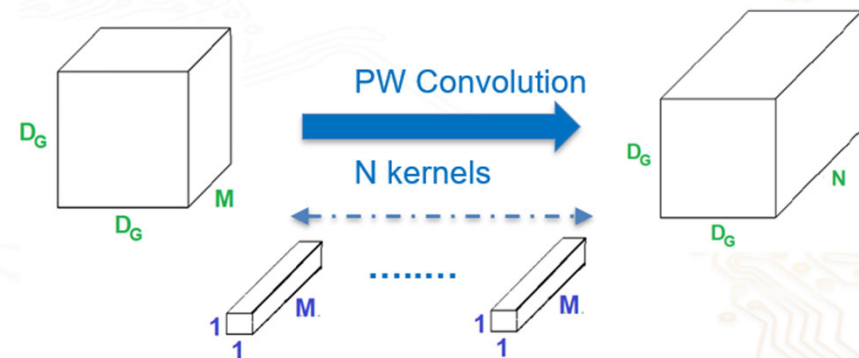


Depthwise Separable Convolution Complexity

$$\text{DWC: } D_G \times D_G \times D_K \times D_K \times M$$



$$\text{PWC: } D_G \times D_G \times M \times N$$



$$\begin{aligned} &\text{Total number of multiplications} \\ &= \text{DWC multiplications} + \text{PWC multiplications} \\ &= D_G \times D_G \times D_K \times D_K \times M + D_G \times D_G \times M \times N \\ &= D_G \times D_G \times M \times (D_K \times D_K + N) \end{aligned}$$

Complexity Comparison

Number of multiplications for conventional CNN

$$= N \times D_K \times D_K \times M \times D_G \times D_G$$

Number of multiplications for DWS CNN

$$= D_G \times D_G \times M \times (D_K \times D_K + N)$$

Ratio of DWS/CNN

$$= D_G \times D_G \times M \times (D_K \times D_K + N) / (N \times D_K \times D_K \times M \times D_G \times D_G)$$

$$= (D_K \times D_K + N) / (N \times D_K \times D_K)$$

$$= 1/N + 1/D_K^2$$

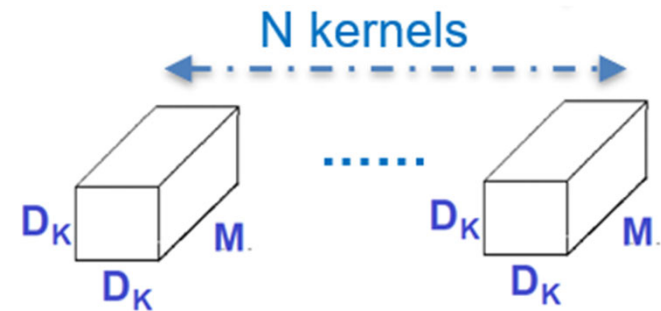
Example: N kernels = 1024, with $D_K = 3$

$$1/N + 1/D_K^2 = 1/1024 + 1/(3^2) = 0.112 \quad (89\% \text{ reduction})$$

Complexity Comparison

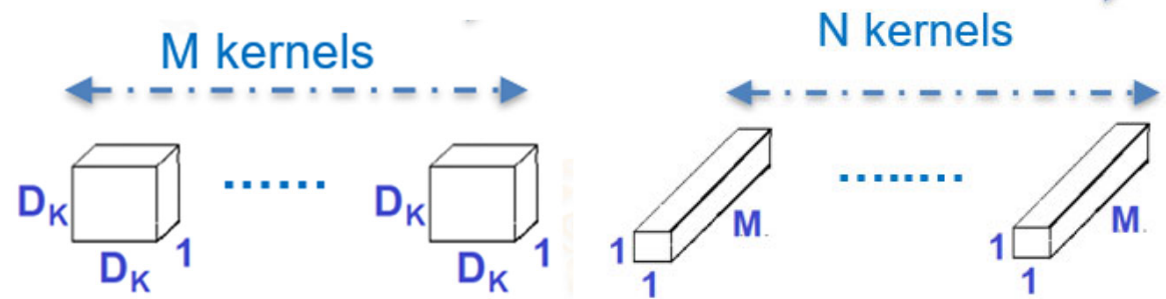
Number of parameters for conventional CNN

$$\begin{aligned} &= (D_K \times D_K \times M) \times N \text{ kernels} \\ &= (D_K^2 \times M \times N) \end{aligned}$$



Number of parameters for DWS CNN

$$\begin{aligned} &= (D_K \times D_K \times M) + (M \times N) \\ &= M \times (D_K^2 + N) \end{aligned}$$



Ratio of DWS/CNN

$$\begin{aligned} &= M \times (D_K^2 + N) / (D_K^2 \times M \times N) \\ &= 1/N + 1/D_K^2 \end{aligned}$$

MobileNets

MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications

Andrew G. Howard Menglong Zhu Bo Chen Dmitry Kalenichenko
Weijun Wang Tobias Weyand Marco Andreetto Hartwig Adam

Google Inc.

{howarda, menglong, bochen, dkalenichenko, weijunw, weyand, anm, hadam}@google.com

MobileNet uses Depthwise Separable Convolution

- to reduce latency and model size
- for computer vision based recognition applications on mobile devices

MobileNet Applications

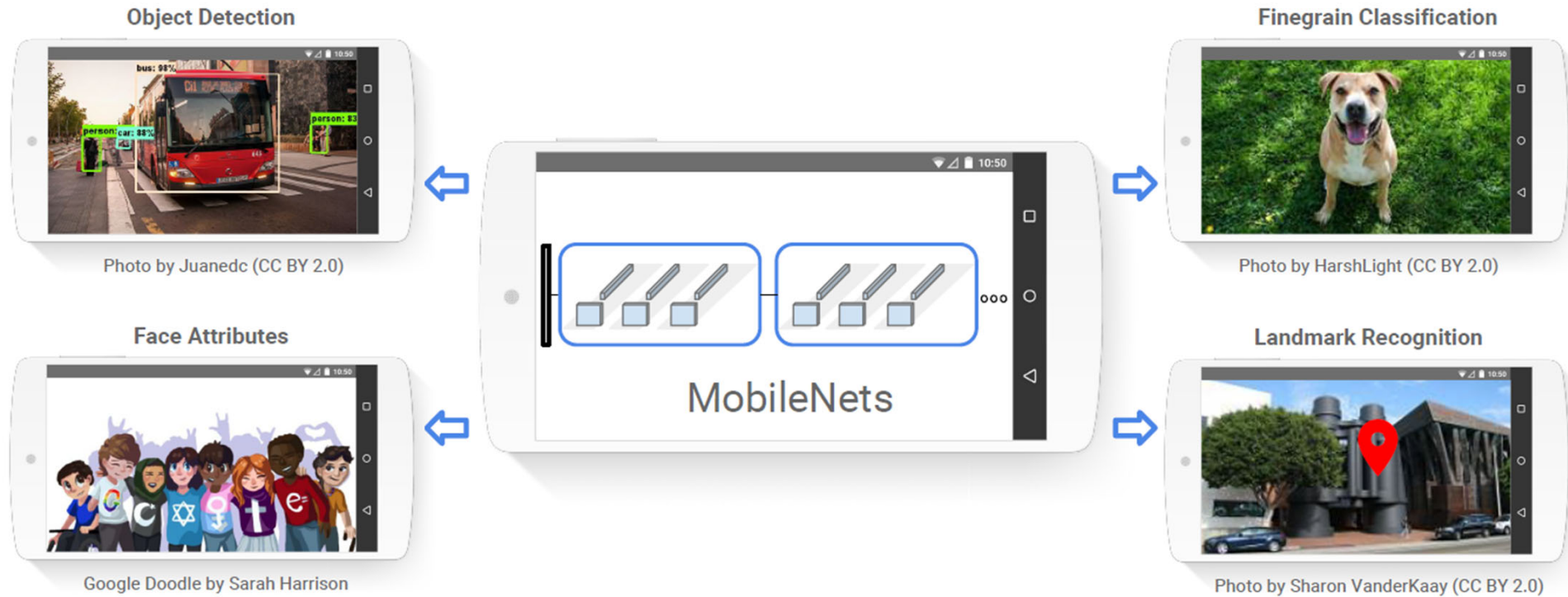


Figure 1. MobileNet models can be applied to various recognition tasks for efficient on device intelligence.

MobileNet Architecture

Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
5×	Conv dw / s1 $3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
	Conv / s1 $1 \times 1 \times 512 \times 512$	$14 \times 14 \times 512$
Conv dw / s2	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
Conv dw / s2	$3 \times 3 \times 1024$ dw	$7 \times 7 \times 1024$
Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$
FC / s1	1024×1000	$1 \times 1 \times 1024$
Softmax / s1	Classifier	$1 \times 1 \times 1000$

Table 4. Depthwise Separable vs Full Convolution MobileNet

Model	ImageNet	Million	Million
	Accuracy	Mult-Adds	Parameters
Conv MobileNet	71.7%	4866	29.3
MobileNet	70.6%	569	4.2

On ImageNet

- accuracy reduces by 1%
- tremendous saving on MAC operation and number of parameters.

MobileNet Adjustable Hyper-Parameters

For specific use case or application that requires the model to be smaller and faster

- two adjustable hyper parameters are introduced in the design
 - i. Width Multiplier α
 - ii. Resolution/Depth Multiplier ρ

Allow the model to be adjusted to be smaller and less computationally expensive models

Layer/Modification	Million Mult-Adds	Million Parameters
Convolution	462	2.36
Depthwise Separable Conv	52.3	0.27
$\alpha = 0.75$	29.6	0.15
$\rho = 0.714$	15.1	0.15

Width Multiplier

The role of the width multiplier α

- to thin/narrow a network uniformly at each layer
- by reducing the number of filters used

For a given layer and width multiplier

- the number of input channels M becomes αM and the number of output channels N becomes αN
- $\alpha \in (0, 1]$ with typical settings of 1, 0.75, 0.5 and 0.25, where 1 is the baseline MobileNet and $\alpha < 1$ are reduced MobileNets.

Table 6. MobileNet Width Multiplier

Width Multiplier	ImageNet Accuracy	Million Mult-Adds	Million Parameters
1.0 MobileNet-224	70.6%	569	4.2
0.75 MobileNet-224	68.4%	325	2.6
0.5 MobileNet-224	63.7%	149	1.3
0.25 MobileNet-224	50.6%	41	0.5

Resolution Multiplier

This parameter ρ is applied to the input image

- make the internal representation of every layer to subsequently be reduced by the same amount.
- equivalent to varying the settings of the input resolution

Accuracy drops off smoothly across resolution from 224 to 128.

Table 7. MobileNet Resolution

Resolution	ImageNet Accuracy	Million Mult-Adds	Million Parameters
1.0 MobileNet-224	70.6%	569	4.2
1.0 MobileNet-192	69.1%	418	4.2
1.0 MobileNet-160	67.2%	290	4.2
1.0 MobileNet-128	64.4%	186	4.2

Summary

There are different approaches that can be used to implement CNN based application

Depthwise Separable Convolution uses two stages of convolutions

- Depthwise convolution
- Pointwise convolution
- reduces computation time and number of parameters

Depthwise Separable Convolution is used in MobileNet

- to build smaller and faster MobileNets
- width multiplier and resolution multiplier are used to trade off a reasonable amount of accuracy to reduce size and latency.